HILABS AIQUEST 2026 · TEAM RANGER

TermMap: mapping messy clinical terms to ICD-10-CM and SNOMED CT

Team Ranger · IIT Madras | Naveen Prakash Junodiya bs23b011@smail.iitm.ac.in · Neeraj Meena ch23b099@smail.iitm.ac.in · Khushwant Meena bs23b029@smail.iitm.ac.in

Open-weight, self-hosted · Deterministic · One command · ▶ Demo video · GitHub · Weights

59.7%
Headline (lower of the two systems), out-of-fold on 9,646 rows
from 36.7% on day 1

72.8%
ICD-10-CM exact match
+30.0 pts vs lexical baseline

69.0%
SNOMED CT on diagnosis rows, which is the assessment's scope
ICD on diagnosis rows: 72.8%

53 / 48%
ICD / SNOMED accuracy on codes **never seen** in training
from 28 / 29% with our first ranker

The short version. We treated this as a **search-then-choose** problem, not as classification. Eight searches (spelling- and meaning-based) shortlist candidate codes from the full dictionaries. Fine-tuned encoders and a LightGBM ranker narrow them to ten. A small open LLM (Qwen2.5-7B), which we fine-tuned with LoRA on the training data, then reads the term, its sentence and similar coded training terms, and adjudicates. All scores are honest 5-fold cross-validation grouped by chart chunk: every model is retrained per fold and never scores rows it saw.

1 Approach: how we read the problem

We spent the first hours only on the data, and three facts shaped everything after that.

45%
Most answers are new
45% of SNOMED rows (23% of ICD) have a gold code absent from the other folds. Memorising term→code pairs cannot work, so we **retrieve from the full dictionaries**: 98k ICD codes and 382k SNOMED concepts.

7,384
Terms are noisy chart text
That many distinct entities cover 5,202 codes, written like *"htn"*, *"h/o breast ca"* or *"chronickidney dis stag 3"*. Spelling-only matching breaks, so we needed **spelling- and meaning-based** search.

min()
The weaker system decides
The headline is the lower of ICD and SNOMED, and SNOMED stayed 13–15 points behind. So most effort went into SNOMED, especially **choosing between close siblings** (finding vs disorder, specified vs unspecified).

How we framed it

- **Retrieve, then rank.** First get the right code into a short list (recall), then pick it (precision). Measuring both separately told us at every step which half to fix.
- **Context is part of the input.** "MS" is multiple sclerosis or mitral stenosis, and "history of" turns a cancer code into a Z-code, so the models that make the final choice always read ±25 words around the term.
- **Honest validation first.** Rows from one chart chunk stay in one fold (GroupKFold, 5 folds), because a random split leaks.

Scope decisions

- **Diagnoses first.** The assessment covers diagnoses, so we report diagnosis-row accuracy too, and at inference the subconcept is fixed to "diagnosis" (the hidden file has only entity + chunk_text).
- **Both systems for every row, no parent-code fallback.** The organisers confirmed there is no partial credit.
- **Open, self-hosted, deterministic.** The data is proprietary, so it never touched an external API. One GPU, no sampling.

2 Architecture: one term, end to end

One **entity + chunk_text** goes in; the pipeline runs once per coding system, each stage passing a shorter list on.

INPUT

"h/o breast ca" + "...s/p lumpectomy 2019, no evidence of recurrence..."

normalised: lower-case, punctuation removed; ±25 context words cut around the term

▼

1 · RETRIEVE

8 searches, merged by reciprocal-rank fusion → top 100 codes

TF-IDF char · dictionary | TF-IDF word · dictionary | TF-IDF char · training terms | SapBERT · dictionary | GTE · dictionary | SapBERT · similar training terms | GTE · similar training terms | **SapBERT v2 fine-tuned (ours)**

spelling-based · meaning-based · our retriever, trained on our data

▼

2 · SCORE

~80 signals for each of the top 41 candidates

string & head-word match | laterality · acuity · history · negation | specificity vs "unspecified" | ICD block votes · code priors | **SapBERT v2 + v3 scores** | **MedCPT cross-encoder (fine-tuned)** | **SNOMED semantic-tag classifier**

▼

3 · RANK A

LightGBM LambdaRank: an ICD model, plus a pooled model used for SNOMED, 3 seeds each → **top 10**

▼

4 · ADJUDICATE

Qwen2.5-7B-Instruct + our LoRA sees the term, ±25 words, 8 similar coded training terms and the 10 options (A–J). It returns a probability per option from a single forward pass, read twice in opposite option orders to cancel position bias.

▼

5 · RANK B

LightGBM again, now with the LLM's probabilities added → final code + confidence margin

example output: ICD **Z85.3** personal history of breast cancer · SNOMED **429087003** history of malignant neoplasm of breast

Correct code still in the list



The gap between "in the top 10" and "chosen" is where the LLM earns its place: it lifts top-1 by +4.5 (ICD) and +4.8 (SNOMED) points. The gap above the top 100 is the retrieval ceiling (section 5).

Out-of-fold everywhere. Every fine-tuned model was trained 5 times, once per fold, so the rankers learn from honest scores. It was then trained once more on all rows for the final system.

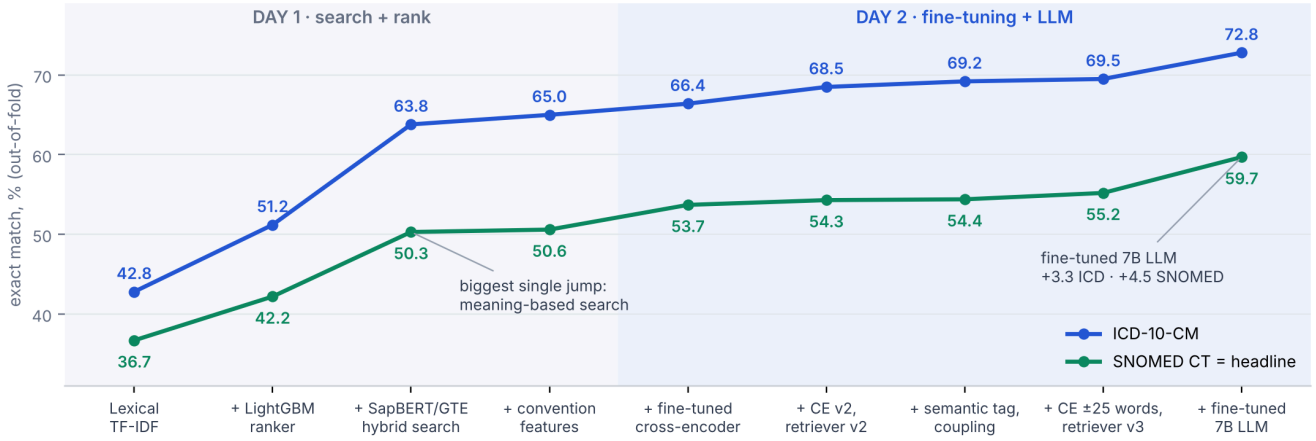
Why two rankers? Ranker A decides what the LLM sees (10 options, one forward pass), and ranker B decides how much to trust it. The LLM never answers on its own.

Where each model sits, and how it was trained

Model / resource	Stage	What we did with it	Compute
TF-IDF (char 3–5-grams, words)	1	Index over 159k ICD + 629k SNOMED descriptions + training terms	CPU
SapBERT, GTE-base	1	Off the shelf, meaning-based search (dictionaries + training terms)	Kaggle T4
SapBERT v2 / v3 fine-tuned	1, 2	Contrastive, self-mined hard negatives; v3 + synonym pairs, chart noise	Kaggle T4×2
MedCPT cross-encoder fine-tuned	2	Listwise vs 11 hard negatives; "entity | ±25 words" vs description	Kaggle T4×2
Semantic-tag classifier	2	Logistic regression: disorder / finding / procedure... from the text	CPU
LightGBM LambdaRank	3, 5	Learning-to-rank; ICD model + pooled SNOMED model, 3 seeds	CPU
Qwen2.5-7B-Instruct LoRA	4	LoRA r=16; loss on the answer letter only; options shuffled	1× RTX 4090

3 How we got here: two days, step by step

Every point on this chart is a full 5-fold, out-of-fold run on all 9,646 rows. We kept a change only if it moved these numbers. Below the chart: what we tried, what broke, and what we did next.



1 · Lexical search first 36.7%

Tried: TF-IDF over the dictionary descriptions, taking the top hit. **Broke:** synonyms and chart shorthand ("htn", "heart attack" vs "myocardial infarction") have no shared characters. Even with a LightGBM ranker on top (42.2%), the right code was simply *not in the list* too often. **Did:** measured recall and ranking separately from then on.

2 · Meaning-based search: the biggest single jump +8.1

Tried: SapBERT and GTE embeddings over the dictionaries, plus "nearest training terms" as a search source, fused with reciprocal rank. Only the *public* dictionaries were embedded on Kaggle, while training terms stayed private. **Result:** recall@50 went from 78/75% to 90/86%, and SNOMED reached 50.3%.

3 · Hand-made features hit a ceiling +0.3

Tried: laterality, acuity, history, negation and specificity features, plus context rules and abbreviation expansion. **Broke:** error analysis showed the gold code was usually *already shortlisted* (median rank 5), and the ranker still picked a close sibling. Rules could not read the sentence. **Learned:** the next gain needs a model that reads text.

4 · A cross-encoder, but only once fine-tuned +3.1

Tried: zero-shot MedCPT cross-encoder, which gave just +0.2. It knows medicine but not *our coders' conventions*. **Did:** fine-tuned it on Kaggle T4s (listwise loss, hard negatives from our own shortlist), 5 times so every score is out-of-fold. Adding the SNOMED semantic tag to the description (v2) and widening the context to ±25 words (v4) each helped.

5 · The subconcept surprise

Broke: our features used the "subconcept" column, but the hidden file has only entity + chunk_text, and without it SNOMED dropped 1.9 points. **Did:** built a classifier that predicts it from text (87% accurate). We then asked the organisers, and they confirmed the assessment is diagnosis-only, so the final system safely fixes subconcept = diagnosis.

6 · Fine-tuned retrievers for the long tail +1.2 ICD

Did: trained SapBERT as our own retriever with model-mined hard negatives (v2). In v3 we added dictionary synonym pairs plus simulated chart noise, so it also learns codes nobody in training used. Recall@40 for unseen SNOMED codes: 78.5%.

7 · The LLM: first it lost, then it won +4.5 SNOMED

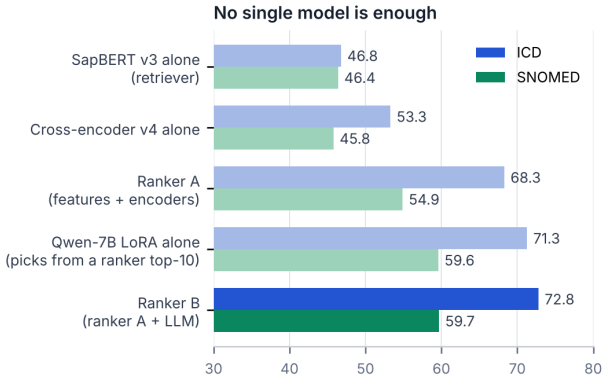
Broke: our first LoRA smoke test (300 examples) was *worse* than the ranker, at 48.7% vs 66.7% on ICD. **Did:** trained Qwen2.5-7B on each full fold (≈50 min per fold on one RTX 4090), scoring with two option orders. It then beat the ranker in every fold: SNOMED 59.6% vs 54.4% on its own, over all 5 folds. Fed into ranker B, it lifted the headline to 59.7%. ICD↔SNOMED coupling, which had helped earlier, stopped helping after this, so we removed it.

The pattern behind the curve. Day 1 gains came from *finding* the right code (recall). Day 2 gains came from *choosing* it, and every big step there was a model that learned **our coders' conventions from the data** through fine-tuning, not a hand-written rule. Rules and features stalled at +0.3, while fine-tuning added +3.1 (cross-encoder) and +4.5 (LLM) on SNOMED.

4 Design decisions, and the evidence behind each

Each choice below was made by an experiment, not by taste. We list what we chose, what we tried instead, and the number that decided it. All figures are out-of-fold exact match (ICD / SNOMED).

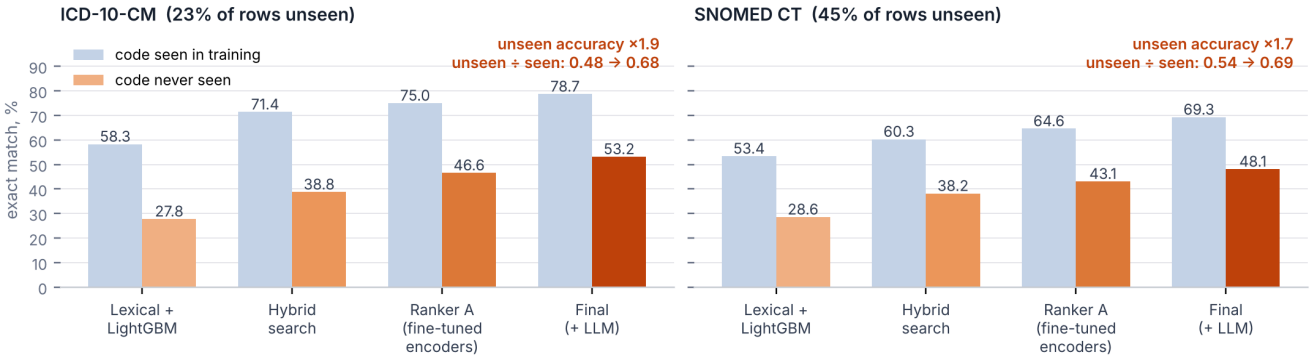
A **Retrieval: hybrid search, fused by rank**	**Chose:** 8 sources (3 spelling-based, 4 meaning-based, 1 fine-tuned retriever) merged by reciprocal-rank fusion. RRF only needs each source's ranking, so a TF-IDF score and a cosine score never have to be calibrated against each other. **Instead of:** a single dense index. **Training terms as a source:** "how our coders wrote it last time" is often the fastest route to the gold code.	RECALL@50 TF-IDF 78 / 75 hybrid **90 / 86** SHORTLIST 41 KEEPS **90.1 / 86.2**
B **Reranker: learning-to-rank on top of every signal**	**Chose:** LightGBM LambdaRank over ~80 features, with the neural scores as features and not as the decision. There is a separate ICD model, and a pooled ICD+SNOMED model scores SNOMED because it learns from more queries. 3 seeds are averaged. **Instead of:** trusting the cross-encoder directly. On its own it reaches only 53.3 / 45.8 (chart).	CROSS-ENCODER ALONE 53.3 / 45.8 AS A RANKER FEATURE **+3.1** SNOMED
C **LLM: small, fine-tuned, shortlist-only, and only a voice**	**Chose:** Qwen2.5-7B with LoRA, choosing among ranker A's top 10 (the gold code is there 88.7 / 82.4% of the time). We read the letter probabilities from one forward pass, twice with the option order reversed to cancel position bias. Ranker B then decides how much to trust it. **Instead of:** a bigger zero-shot model (we tried Qwen2.5-72B, but GPU capacity ran out), or letting the LLM answer alone. **Why small:** fine-tuning on our conventions beat size, and 7B fits one GPU.	LLM ALONE 71.3 / 59.6 RANKER B **72.8 / 59.7** DETERMINISTIC no sampling
D **Specificity policy: learned from our coders, not hard-coded**	**Chose:** no global "always specific" or "always unspecified" rule. Detail words in the term or context (left/right, stage, acute/chronic, "with") are features against "unspecified" codes. ICD-block voting tells the ranker which category the evidence points to. The LLM is told: *"Choose an unspecified code only when the text supports nothing more specific"*, and sees 8 similar coded training terms, so it copies the local convention. **No parent-code fallback**, because there is no partial credit.	ICD ERRORS IN THE RIGHT CATEGORY 43.9% (STILL THE #1 GAP, §5)
E **Context window: ±25 words, read by the models that choose**	**Chose:** the cross-encoder and the LLM read the term plus ±25 surrounding words. The lexical retrievers read only the term, which keeps recall stable. **Instead of:** ±8 words (our first cross-encoder) or hand-written context rules, which gave no gain. History, negation, laterality and "MS = mitral stenosis" all live in the context.	±8 → ±25 WORDS **+0.5** SNOMED CONTEXT RULES ±0.0
F **Validation: leak-free stacking, checked end to end**	**Chose:** GroupKFold by chart chunk (5 folds). Every neural model was trained 5× so its scores are out-of-fold, and the LLM's in-context examples come only from other folds. Finally, a held-out fold ran through the exact `predict.py` code path, with all models trained without it. **Instead of:** a random split, which leaks chunk-mates into training.	PREDICT.PY VS CV 67.9 vs 67.9 58.5 vs 58.6 **98.9%** same



Why stack instead of picking a winner? Each model sees something different. The retriever knows names, the cross-encoder reads the term next to each description, the features encode conventions, and the LLM reasons over the context and similar coded examples. None of them alone reaches the headline. Stacking them, each with honest out-of-fold scores, does: ranker B is better than every model on its own, in both systems.

5 Long-tail generalization: does it reason, or memorise?

We call a row **unseen** when its gold code appears nowhere in the other folds, so that code cannot be learned from training pairs. That covers **45% of SNOMED rows** (46.5% of SNOMED diagnosis rows) and 23% of ICD rows. A memorising system would score close to zero on them.



Out-of-fold exact match split by whether the gold code was seen in training. Accuracy on unseen codes grew faster than on seen ones, so the gap closed in both systems.

53.2%
ICD accuracy on codes never seen in training (first ranker: 27.8%)

58.6%
SNOMED accuracy on **unseen diagnosis** codes, the assessment's scope

279 vs 113
unseen SNOMED rows the LLM **fixed** vs broke, compared with ranker A

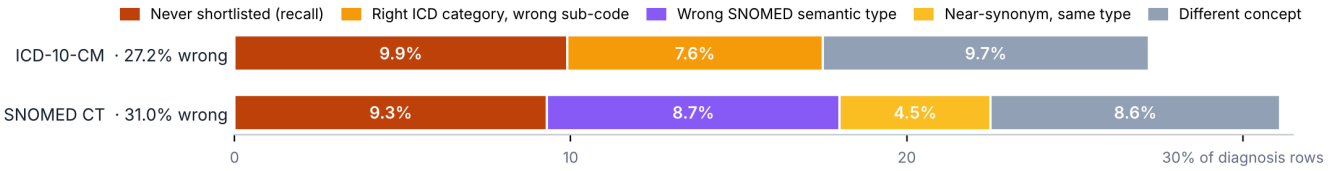
What makes it generalize: five mechanisms

Mechanism	How it helps codes nobody in training used	Evidence
1. Retrieve from the full dictionary	Candidates come from all 98k ICD codes and 382k SNOMED concepts, not from a fixed label set, so any code can be predicted.	gold in top 41 for 79.5% of unseen SNOMED
2. Retriever trained on the dictionary itself	SapBERT v3 also trains on pairs of two names of the *same* code taken from the dictionaries, with typos, dropped spaces and acronyms simulated. It learns every code, not only the 4k in our rows.	unseen recall@40: ICD 66→76, SNOMED 69→76 (fold 0, base → v3)
3. Read descriptions, not IDs	The cross-encoder and the LLM compare the term with the code's *description*, plus the SNOMED semantic tag or the ICD category name, so they need no history for that code.	LLM: unseen ICD +6.6, unseen SNOMED +5.0 over ranker A
4. Borrow conventions from neighbours	For a new term the LLM sees 8 similar coded training terms. Even when the exact code is new, the *habit* carries over (e.g. history → Z-code, laterality → specific code).	overall unseen ÷ seen accuracy: 0.54 → 0.69 (SNOMED)
5. Priors cannot dominate	Code frequency priors are only one of ~80 features and are computed out-of-fold, so the ranker learns how far to trust "popular" codes on new terms.	no drop on seen codes: 64.6 → 69.3

Where the long tail still fails. For unseen codes, recall rather than choosing is the limit. The gold code reaches the shortlist for 92–94% of seen rows but only **78–80% of unseen rows**, and a code that is never retrieved cannot be picked. Typical misses are rare concepts written very differently from any dictionary description, and heavy abbreviations. Our plan for that gap is in section 6.

6 Error analysis: where it still fails, and why

We took every wrong prediction on the diagnosis rows (the assessment's scope) and sorted it by *what went wrong*. "Never shortlisted" is a search failure. Everything else is a choosing failure: the right code was in our top 41 and a neighbour won.



Failure mode	Share	Why it happens	What would fix it
Never shortlisted	9.9 / 9.3%	Rare concepts worded unlike any dictionary description; heavy abbreviations. Mostly unseen codes (51% ICD, 71% SNOMED).	Synonym index, LLM query rewriting
Right ICD category, wrong sub-code	7.6%	Too vague (chose "unspecified") 2.5%, too specific 1.4%, wrong sibling 3.7%; the coders' habit decides.	Show the LLM parent + all siblings
Wrong SNOMED semantic type	8.7%	Same idea, wrong kind: disorder ↔ finding 3.4%, morphology ↔ disorder/finding 1.2%.	Decide the type first, then the concept
Near-synonym, same type	4.5%	Heavily overlapping descriptions; some are equally valid to a human.	Route to a coder via confidence
Different concept	9.7 / 8.6%	Misread term, often a short acronym. Half the ICD cases stay in the right chapter.	Wider context, stronger / ensembled LLM

Real examples (out-of-fold predictions, i.e. made by models that never saw these rows)

System	Term	Gold	Our prediction
Right category, wrong sub-code (ICD)			
ICD	*"100% mlad"*	I25.82 Total occlusion of coronary artery	I25.10 Atherosclerotic heart disease of native coronary artery without angina pectoris
ICD	*"active withdrawal"*	F19.939 Other psychoactive substance use, unspecified with withdrawal, unspecified	F19.930 Other psychoactive substance use, unspecified with withdrawal, uncomplicated
Wrong semantic type (SNOMED)			
SNOMED	*"aaa s/ evar"*	139941000119104 History of endovascular stent graft for repair of abdominal aortic aneurysm (situation)	444588006 Endovascular repair of abdominal aortic aneurysm (procedure)
SNOMED	*"abdominal or pelvic mass"*	860713000 No mass of abdomen	271860004 Abdominal mass
Never shortlisted			
ICD	*"1-2+ ai"*	I35.1 Nonrheumatic aortic (valve) regurgitation NOS	I34.0 Nonrheumatic mitral (valve) insufficiency
SNOMED	*"100% af"*	440059007 Persistent atrial fibrillation (disorder)	49436004 Atrial fibrillation

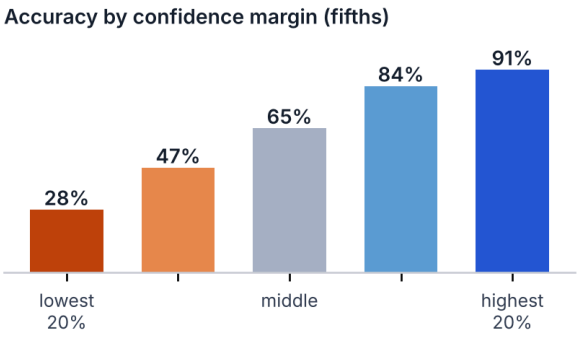
What these examples teach. Often the deciding signal sits outside the term. For *"abdominal or pelvic mass"* the coder chose "**no** mass" (negation), and "s/" in *"aaa s/ evar"* means status post (a situation, not a procedure). Sometimes the term itself misleads: "ai" is aortic insufficiency, but the dictionary says "aortic regurgitation", so search never links them.

7 Closing the gap: confidence, the LLM's trade-off, and our plan

What the LLM fixes, and what it breaks

Compared with ranker A on diagnosis rows, the LLM stage **fixed 361** errors and **introduced 150**, a net gain of +211 rows. Its fixes swap a far-off neighbour for the right concept, while its own mistakes are mostly close calls (36% of its ICD breaks are sibling sub-codes within the right category).

Sys	Term	Ranker A said	Final says (correct)
SNOMED	*"1-2+ ai"*	A 58533008 Artificial insemination	B 60234000 Aortic valve insufficiency
SNOMED	*"30% focal mrca stenosis"*	A 26900001 Coronary ostium stenosis	B 1254658009 Stenosis of mid portion of right coronary artery



The margin between the top two codes is a **usable confidence**: 91% correct in the most confident fifth of rows, 28% in the least confident fifth. `predictions_debug.csv` carries it for every row.

Deployable today: route the least confident rows to a coder

Rows sent to human review	ICD auto accuracy	ICD errors caught	SNOMED auto accuracy	SNOMED errors caught
none (fully automatic)	72.8%	–	59.7%	–
least confident 10%	77.2%	25%	63.7%	19%
least confident 20%	**82.1%**	**47%**	**68.0%**	**37%**
least confident 30%	86.0%	64%	72.1%	52%

Out-of-fold, all rows. One coder reviewing a fifth of the rows would catch almost half of all ICD mistakes.

Plan for the remaining gap (cheapest first)

These are next steps, not yet built. Each one targets a failure mode measured in section 6.

#	Next step	What it targets, and why we believe it	Errors in reach (diagnosis rows)
1	**Hierarchy-aware LLM prompt**	Show the parent code and *all* its siblings (e.g. every I25.8x), so the specificity choice is made between true alternatives instead of whatever the ranker happened to shortlist.	7.6% ICD
2	**Type-first SNOMED decision**	Decide disorder vs finding vs situation first (our tag classifier is already a feature), then choose the concept inside that type. This targets the single biggest SNOMED choosing error.	8.7% SNOMED
3	**Show the LLM 20 options, not 10**	The gold code is in the shortlist of 41 for 86.2% of SNOMED rows but in the top 10 for only 82.4%. Those 3.8 points are invisible to the LLM today.	3.8% SNOMED 1.4% ICD
4	**Query rewriting for acronyms**	Let the LLM expand "ai", "mlad", "s/" before search, and add a synonym-rich index such as UMLS (licence permitting). This is the only way to reach codes that are never shortlisted.	9.3–9.9% (recall)
5	**Whole-chunk context + bigger / 2nd LLM**	Negation and history can sit beyond ±25 words. A second LoRA seed or a 14–72B model, which we started but could not finish for lack of GPU time, would add ensemble stability.	general

The honest ceiling. With today's retrieval, a perfect chooser would reach 90.1% ICD and 86.2% SNOMED. We are at 72.8% / 59.7%, so most of the remaining headroom (about 26 SNOMED points) is in *choosing*, which is exactly where steps 1–3 aim. Recall (step 4) caps everything above that.

8 Innovations

1 · A fine-tuned small LLM inside a learning-to-rank stack

The LLM does not generate an answer. It reads the term, the context and 8 coded neighbours, and gives a probability to each of 10 options in one forward pass, read twice with the order reversed to cancel position bias. A ranker then decides how much to trust it. The result is deterministic, costs one GPU pass, and is worth +4.5 / +4.8 points.

2 · Leak-free stacking, including the LLM

Every neural component (two retrievers, the cross-encoder, the LLM) was trained 5 times so the rankers only ever see out-of-fold scores. Even the LLM's few-shot examples come only from other folds. A held-out run through `predict.py` matches cross-validation to within 0.1 points.

3 · Training terms as a second dictionary

How our coders already wrote and coded similar terms is used three ways: as a retrieval source, as nearest-neighbour votes, and as the LLM's in-context examples. Local conventions get learned without writing a single rule.

4 · A retriever built for the long tail

SapBERT fine-tuned with self-mined hard negatives, plus dictionary synonym pairs and simulated chart noise (typos, run-together words, acronyms), so it learns all 480k codes and not only the 5k we have labels for. Unseen-code accuracy rose ×1.9 (ICD) and ×1.7 (SNOMED).

5 · Confidence that is actually calibrated

The ranker margin ships with every prediction. Sending the least confident 20% to a coder lifts automatic ICD accuracy from 72.8% to 82.1%: a human-in-the-loop design that can be deployed today.

6 · Privacy by construction

Open-weight models only, all self-hosted, and no data ever sent to an external API. Early on, only the public dictionaries went to a cloud GPU, while clinical terms were embedded locally.

9 Challenges & learnings

What didn't work

Idea	SNOMED effect
Hand-written context rules (negation, history)	±0.0
Abbreviation-expansion dictionary	±0.0
Zero-shot cross-encoder (no fine-tuning)	+0.2
Up-weighting / training on diagnosis rows only	±0.2
ICD↔SNOMED coupling once the LLM was in	−0.2
LLM trained on 300 examples (smoke test)	worse than ranker
Qwen2.5-72B zero-shot	not finished: GPU capacity

Engineering challenges we hit

- The CPU sandbox killed jobs after ~2 h, so the runner now checkpoints every fold.
- A paused GPU could not be resumed with the final LoRA inside, so we re-trained it on a new GPU from the same recipe and seed; the loss curve matched.
- Two spot H200s vanished at start-up, so we dropped the 72B run.

What we learned

- **Fine-tuning beat model size and rules.** A 7B model tuned on our conventions beat everything hand-built. The ±0.0 rows on the left are the proof.
- **Measure recall and choosing separately.** Knowing whether the gold code was retrieved told us what to work on next.
- **Stack, don't pick.** No single model reaches the headline.
- **Small samples lie.** The LLM looked useless on 300 examples and was the biggest win on 7.7k.

What we'd do differently

- Build the leak-free out-of-fold harness on hour one. Everything after it became trustworthy.
- Start the LLM on day 1, not day 2: it was the largest single lever.
- Ask the organisers about scope (subconcept, partial credit) before building features around it.
- Sync trained artefacts off rented machines automatically.

10 Engineering, reproducibility, cost and resources

One command	`python predict.py --input hidden.csv --out predictions.csv` (add `--no-llm` on a CPU-only machine, which returns ranker A's answer). It was run end to end on a rented RTX 4090.
Deterministic	No sampling anywhere. The LLM is read as letter probabilities, ties break by code string, LightGBM runs with `deterministic=True`, and all seeds are fixed.
Speed and cost per term	Two 7B forward passes (~900-token prompt, one output position) plus small encoders. Measured on one RTX 4090: 300 term–system pairs in 290 s including model loading (under 1 s each), i.e. ≈ ₹0.02 at ₹69/hour, with no API cost. Batching with vLLM would cut this several-fold at chart volume.
Training cost	LLM: about 6 GPU-hours on one rented RTX 4090 (5 folds + full). Encoders: free Kaggle T4×2. Rankers: CPU.
External resources	SapBERT (cambridgeltl/SapBERT-from-PubMedBERT-fulltext) · GTE-base (thenlper/gte-base) · MedCPT Cross-Encoder (ncbi/MedCPT-Cross-Encoder) · Qwen2.5-7B-Instruct (Apache-2.0) · LightGBM · scikit-learn · transformers · PEFT · sentence-transformers · rapidfuzz. Ontologies: the ICD-10-CM and SNOMED CT US code files provided by HiLabs. No crosswalk, UMLS or other external dataset was used.
Links	Code: github.com/naveentheog/termmap · Weights: huggingface.co/Naveenmeena/termmap-weights · Demo video: Google Drive (demo video)

Reproduce it end to end

#	Step	Command / where	Time
1	Install dependencies	`pip install -r requirements.txt`	~2 min
2	Data + weights	HiLabs files → `data/` ; `python tools/download_weights.py` pulls our models + embeddings from Hugging Face	~5–15 min
3	(only if embeddings are missing)	`python tools/build_dict_embeddings.py` (GPU)	~10 min on a 4090
4	Predict	`python predict.py --input hidden.csv --out predictions.csv`	<1 s / term
5	Verify	`python tests/test_final_heldout.py` : held-out fold through the predict path vs CV	~15 min CPU
–	Re-train everything	`kaggle/` (retrievers, cross-encoder), `llm/` (LoRA), `experiments/run_fast.py` , `train_final.py`	~10 GPU-h total

Checks we ran before submitting

98.9%
Same predictions as CV
Held-out fold 0, every model trained without it, through the exact `predict.py` code path: ICD 67.9 vs 67.9, SNOMED 58.5 vs 58.6.

GPU run
End to end on a real GPU
All six neural models loaded and run on one RTX 4090; demo cases (abbreviation collision, OCR-mangled term, specificity, history) came out right.

0 keys
No secrets, no external APIs
Nothing needs an API key. `.env.example` lists only optional settings, and `data/` , `weights/` and `.env` are git-ignored.

In one line. TermMap retrieves from the full dictionaries, learns our coders' conventions through fine-tuning, and lets a small, deterministic LLM make the last call. Every number in this write-up is honest out-of-fold, and the same code path produces the submitted predictions.